

Diseño Físico y Edición de Datos

[Descargar estos apuntes](#)

Índice

- ▼ Diseño Físico y Edición de Datos
 - [Del diseño lógico al diseño físico](#)
 - [Crear Base de Datos](#)
 - [Tablas y campos](#)
 - ▼ [Añadir y editar registros](#)
 - [INSERT](#)
 - [UPDATE](#)
 - [DELETE](#)
 - [Índices](#)
 - ▼ [Restricciones](#)
 - [Restricción de campo clave](#)
 - [Restricción de campo obligatorio](#)
 - [Restricción de campo con valores únicos](#)
 - [Restricción de campo por clave ajena](#)
 - [Restricción por borrado o actualización de una clave ajena](#)
 - [Otras restricciones y valores por defecto](#)
 - [Restricciones avanzadas](#)
 - [Tablas a partir de consultas](#)
 - ▼ [Vistas](#)
 - [Vistas para aplicar restricciones](#)
 - [Vistas para aplicar transformaciones o mostrar campos calculados](#)
 - [Vistas con phpMyAdmin](#)

Del diseño lógico al diseño físico

El **diseño físico** es el proceso de producir la descripción de la implementación de la base de datos en memoria secundaria; estructuras de almacenamiento y métodos de acceso que garanticen un acceso eficiente a los datos.

Para llevar a cabo esta etapa, se debe haber decidido cuál es el SGBD que se va a utilizar, ya que el esquema físico se adapta a él.

En nuestro caso utilizaremos MySQL, y como su sintaxis se adapta en gran medida a SQL ANSI, los comandos que usaremos serán muy similares en el resto de SGBD relacionales.

Cuando llegamos a este punto disponemos de:

- descripción del sistema de información
- el esquema conceptual del modelo entidad-relación
- el esquema del diccionario de datos

- el esquema relacional del modelo relacional

Mientras que en el diseño lógico se especifica qué se guarda, en el diseño físico se especifica cómo se guarda. Para ello, el diseñador debe conocer muy bien toda la funcionalidad del SGBD concreto que se vaya a utilizar y también el sistema informático sobre el que éste va a trabajar.

El diseño físico no es una etapa aislada, ya que algunas decisiones que se tomen durante su desarrollo, por ejemplo para mejorar las prestaciones, pueden provocar una reestructuración del esquema lógico.

El objetivo de esta etapa es producir una descripción de la implementación de la base de datos en memoria secundaria. Esta descripción incluye las estructuras de almacenamiento y los métodos de acceso que se utilizarán para conseguir un acceso eficiente a los datos.

El objetivo por tanto es:

- Conocer las **utilidades o herramientas** para la definición de información en un SGBD.
- Conocer las **órdenes SQL** para la definición de datos.
- Elegir **el tipo de dato** más adecuado para cada tipo de información.
- Definir las **estructuras físicas** de almacenamiento e implantar todas las **restricciones** reflejadas en el diseño lógico (clave primaria, clave alternativa, clave ajena...).

En esta unidad didáctica estudiaremos las instrucciones o comandos SQL del **lenguaje de definición de datos (LDD)** que permiten la creación de nuestra base de datos y que incluirá las tablas con sus campos e índices.

La primera fase del diseño lógico consiste en traducir el esquema lógico global en un esquema que se pueda implementar en el SGBD escogido. Para ello, es necesario conocer toda la funcionalidad que éste ofrece. Por ejemplo, el diseñador deberá saber:

- Si el sistema soporta la definición de claves primarias, claves ajenas y claves alternativas.
- Si el sistema soporta la definición de datos requeridos (es decir, si se pueden definir atributos como no nulos).
- Si el sistema soporta la definición de dominios.
- Si el sistema soporta la definición de reglas de negocio (es decir, restricciones de cardinalidad o de cálculo)
- Cómo se crean las tablas base

SINTAXIS

Utilizaremos:

- palabras en mayúsculas para las reservadas de MySQL
- palabras en minúsculas para los nombres de nuestros elementos (tablas, campos, índices, ...)
- corchetes para indicar un contenido opcional

Para poder conectar a la BD de MySQL vamos a utilizar dos herramientas:

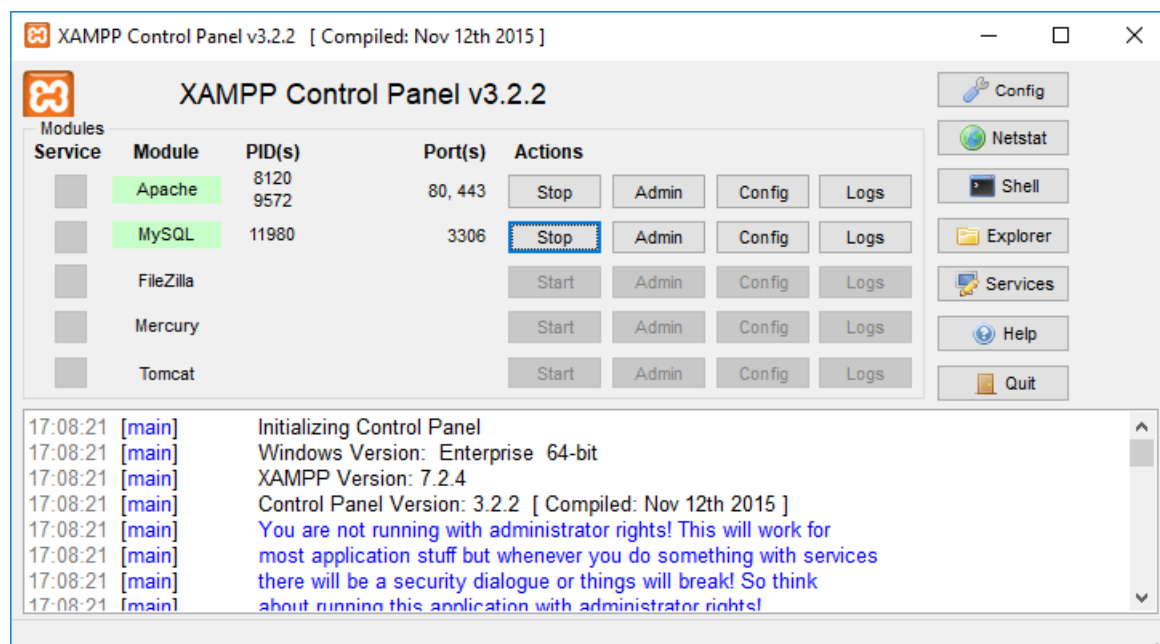
- **Por línea de comandos:** `mysql.exe`
- **Mediante la aplicación web (AW)** `phpMyAdmin`

Las instrucciones que vamos a ejecutar funcionan en las dos herramientas, aunque **phpMyAdmin** proporciona mucha más información de forma gráfica.

Una vez instalado **xampp** abriremos el programa `c:\xampp\xampp-control.exe`.

Desde esta herramienta pondremos en marcha el servidor MySQL. Además, si vamos a usar **phpMyAdmin** tendremos que arrancar también el servicio web de **Apache**.

Abriremos las dos herramientas a la vez y comprobaremos que al ejecutar las instrucciones en una de ellas, su resultado se puede ver en las dos.



Para poder conectar con `mysql.exe` abriremos una ventana de comandos (CMD) y nos desplazaremos hasta la carpeta donde tenemos la herramienta. Para conectar debemos indicar que conectamos con el usuario root que no tiene contraseña:

```
C:> cd \xampp\mysql\bin
C:\xampp\mysql\bin> .\mysql.exe -u root
```

Para conectar con la Aplicación web phpMyAdmin abriremos un navegador e introduciremos la URL:
<http://localhost/phpmyadmin>

Para ejecutar instrucciones usaremos la pestaña SQL.

Crear Base de Datos

Lo primero que debemos hacer es crear una base de datos.

MySQL permite disponer de varias bases de datos en cada instancia que estemos ejecutado. Cuando conectamos a MySQL podemos conocer las bases de datos que hay creadas con el comando:

```
SHOW DATABASES;
```

Para crear una base de datos nueva utilizaremos el comando:

```
CREATE DATABASE NombreBD;
```

Podemos añadir otros parámetros por si no son los que MySQL tiene por defecto:

```
CREATE DATABASE NombreBD
DEFAULT CHARACTER SET utf8
DEFAULT COLLATE utf8_general_ci;
```

En `my.ini` se puede indicar en una variable el **ENGINE** por defecto.

```
[mysqld]
default-storage-engine = InnoDB
```

Por defecto ya tiene este valor y podemos comprobarlo en phpMyAdmin en la pestaña *Variables*.

Para eliminar una base de datos nueva utilizaremos el comando:

```
DROP DATABASE NombreBD;
```

Tablas y campos

Para crear una tabla en el DF (Diseño Físico) partiremos del esquema relacional, el diccionario de datos y la documentación de restricciones.

Debemos buscar la información de restricciones de algunos campos:

- **Restricciones por valor**

Se definen en el diccionario de datos. Estas restricciones pueden estar referidas al:

- Tipo de dato
- Longitud
- Dominio de valores
- Posibilidad de estar vacío (valor nulo) o no
- Valor único (no repetido)

- **Restricciones de existencia**

Se definen en el esquema lógico y se detallan en el diccionario de datos. Un campo que es clave ajena de una tabla en la relación puede ser opcional (tener valores nulos) o no.

- **Restricciones de cardinalidad**

Se definen en la documentación de restricciones.

Ejemplo: Cardinalidad máxima definida = Un empleado en una empresa pertenece a varios departamentos, pero como máximo a 4.

- **Restricciones por campos calculados**

Se definen en el diccionario de datos. El valor de un campo vendrá dado por una fórmula.

Las restricciones por valor y de existencia tendrán efecto en la propia creación de la tabla, pero las restricciones de cardinalidad y por campos calculados serán definidos por **disparadores (triggers)** que comprueben o calculen los datos y así, garanticen su integridad.

El **esquema relacional (modelo lógico) consta de un conjunto de relaciones (tablas)** y, para cada una de ellas, se tiene:

- El **nombre de la relación** (que se convierte en el DF en una tabla)

- La **lista de atributos** entre paréntesis (que se convierten en el DF en campos)
- La **clave primaria y las claves ajenas**, si las tiene.
- Las **reglas de integridad** de las claves ajenas (valor obligatorio o no)

En el diccionario de datos se describen los atributos y, para cada uno de ellos, se tiene:

- Su dominio: tipo de datos, longitud y restricciones de dominio.
- Si puede ser vacío (es decir, si admite valores nulos).
- Si debe tener valor único para cada registro
- El valor por defecto (que es opcional).
- Si es calculado, cómo se calcula su valor.
- Si es enumerado, los valores que puede tener (por ejemplo, SI y NO)

Antes de comenzar necesitaremos conocer algunos tipos de datos de MySQL. Los más utilizados se muestran en la siguiente tabla:

Tipos de datos en MySQL	
Tipo de Dato	Valores
VARCHAR	Cadena de caracteres
INT	Número entero
DECIMAL	Números decimales
DATE	Fecha
DATETIME o TIMESTAMP	Fecha y hora
ENUM	Conjunto de valores

Para las cadenas de caracteres y los números se puede indicar entre paréntesis cuántos caracteres o dígitos utilizaremos. Además, para el tipo de datos DECIMAL se puede indicar cuántos decimales tomamos. Por ejemplo: VARCHAR(5) es una cadena de caracteres de 5 letras, INT(4) un entero de 4 dígitos y DECIMAL(5,2) es un real de 5 dígitos de los cuales 2 son decimales.

Referencias

[Tipos de datos en MySQL – Documentación oficial](#)

Para crear una tabla utilizaremos el comando sql **CREATE TABLE** reducido:

```
CREATE TABLE NombreTabla (
    Campo1,
    Campo2,
    ...
    CampoN
);
```

La sintaxis para definir un **campo** en la instrucción anterior es:

```
nombre_col TIPO
[NOT NULL | NULL]
[DEFAULT valor_por_defecto]
[AUTO_INCREMENT]
[[PRIMARY] KEY]
[UNIQUE]
[COMMENT 'string']
```

Ejemplo 1

Crear una tabla llamada **artículos** con los siguientes campos:

- **codigo** como cadena de 4 caracteres
- **descripcion** como cadena de 100 caracteres
- **precio** como un real con 2 decimales.

```
CREATE TABLE articulos (
  codigo VARCHAR(4),
  descripcion VARCHAR(100),
  precio DECIMAL(10,2)
);
```

Referencias

[CREATE TABLE en MySQL – Documentación oficial](#)

Con el comando **DROP TABLE** podemos eliminar una tabla.

Ejemplo 2

Eliminar una tabla

Eliminar la tabla proveedores.

```
DROP TABLE proveedores;
```

Con el comando **RENAME TABLE** podemos cambiar el nombre de una tabla.

Ejemplo 3

Cambiar el nombre a una tabla

Cambiar el nombre de la tabla proveedores por distribuidores.

```
RENAME TABLE proveedores TO distribuidores;
```

Añadir un campo a una tabla existente

Una vez creada una tabla, es posible que necesitemos añadir, modificar o eliminar algún campo existente. Para ello utilizaremos el comando `ALTER TABLE`. Este comando admite muchas variantes.

Ejemplo 4

Añadir campos a una tabla

Añadir el campo `descuento`, de tipo entero con 3 dígitos y sin aceptar valores nulos, después del campo `precio`.

```
ALTER TABLE articulos
ADD COLUMN descuento INT(3) NOT NULL
AFTER precio;
```

Modificar un campo a una tabla existente

Hay dos formas de cambiar un campo de una tabla:

- **Con la opción `MODIFY`**. Se puede cambiar la definición del campo pero no el nombre.

Peligro

La instrucción `ALTER TABLE` con la opción `MODIFY` hace que se pierdan los datos que había en la columna cambiada.

- **Con la opción `CHANGE`**. Se puede cambiar tanto la definición como el nombre del campo. Con esta opción se preservan los cambios.

Ejemplo 5

Modificar la definición de los campos de una tabla

Modificar el campo `descuento` de manera que ahora pase a tener 4 dígitos.

```
ALTER TABLE articulos
MODIFY COLUMN descuento INT(4) NOT NULL;
```

Ejemplo 6

Modificar la definición de los campos de una tabla

Modificar el campo `descuento` de manera que ahora pase a tener 4 dígitos y se llame `dto`.

```
ALTER TABLE articulos CHANGE descuento dto INT(4) NOT NULL;
```

Eliminar un campo de una tabla existente

Con la instrucción `ALTER TABLE` y la opción `DROP COLUMN` podemos eliminar un campo de una tabla.

Eliminar campos de una tabla

```
ALTER TABLE articulos  
DROP COLUMN dto;
```

Añadir y editar registros

INSERT

El comando **INSERT** inserta nuevas filas en una tabla existente. Existen dos maneras de insertar registros en una tabla:

- Con la cláusula **VALUES**

El formato del comando es **INSERT ... VALUES** e inserta filas basándose en los valores especificados explícitamente.

- **Insertar un registro:** la sintaxis para insertar un registro es la siguiente:

```
INSERT INTO nombreTabla (col_name1, col_name2, ...)  
VALUES (valor_col1, valor_col2, ... );
```

- **Insertar varios registros:** la sintaxis para insertar varios registros es la siguiente:

```
INSERT INTO nombreTabla (col_name1, colname2, ...)  
VALUES  
    (valor1_col1, valor1_col2, ... ),  
    (valor2_col1, valor2_col2, ... ),  
    ...  
    (valorn_col1, valorn_col2, ... );
```

- Con sentencia **SELECT**

El formato del comando es **INSERT ... SELECT** e inserta filas seleccionadas de otra tabla o tablas.

La sintaxis para consultar los registros de una tabla es:

```
SELECT col_name1, col_name2, ...  
FROM nombreTabla  
[WHERE condicion];
```

La sintaxis para **insertar registros desde una consulta SELECT** es la siguiente:


```
INSERT INTO nombreTabla2 (col_name1, col_name2, ...)
SELECT col_name1, col_name2, ...
FROM nombreTabla1
[WHERE condicion];
```

Importante

Si hay campos en la tabla que no se especifican en la instrucción **INSERT** , se le asignará el valor que se haya indicado en su creación por defecto (**DEFAULT**) y en caso contrario será **NULL** .

Ejemplo 1

Insertar valores en tablas

Inserta un teclado y un ratón inalámbrico en la tabla **articulos**.

```
INSERT INTO articulos (codigo, descripcion, precio) VALUES
('0001', 'TECLADO INALÁMBRICO', 10.45),
('0002', 'RATÓN INALÁMBRICO', 7.95);
```

UPDATE

La sentencia **UPDATE** actualiza columnas de filas existentes de una tabla con nuevos valores.

La cláusula **SET** indica las columnas a modificar y los valores que deben tomar.

La cláusula **WHERE** , si se proporciona, especifica qué filas deben ser actualizadas. Si no se especifica, serán actualizadas todas ellas.

La sintaxis para actualizar registros es:

```
UPDATE nombreTabla
SET campo = valor [, campo2 = valor2, ... campoN = valorN]
[WHERE condicion];
```

En la cláusula **SET** utilizaremos *valor* para indicar una cadena, número, fecha, ...

Ejemplo 1

Actualización de registros

Actualizar el nombre de una persona identificada por el DNI.

```
UPDATE personas
SET nombre = 'Juan López'
WHERE dni = '21555666';
```

Ejemplo 2

Actualización de registros

Aumentar la edad de una persona por su cumpleaños.

```
UPDATE personas
SET edad = edad + 1
WHERE dni = '21555666';
```

Ejemplo 3

Actualización de registros

Aumentar el precio de todos los artículos un 5%.

```
UPDATE articulos
SET precio = precio + precio * 5 / 100;
```

Peligro

Cuidado con la cláusula **UPDATE** ya que si no ponemos ninguna condición (cláusula **WHERE**) se actualizarán todos los registros de la tabla.

DELETE

La sentencia **DELETE** elimina las columnas de una tabla que cumplan la condición dada por la cláusula, y devuelve el número de registros borrados. Sin cláusula **WHERE** elimina todos los registros.

La sintaxis para eliminar registros es:

```
DELETE FROM nombreTabla
[WHERE condicion];
```

Peligro

Cuidado con la cláusula **DELETE** ya que si no ponemos ninguna condición (cláusula **WHERE**) se eliminarán todos los registros de la tabla.

Índices

Los índices son estructuras que se crean en las Bases de Datos para poder controlar la integridad y realizar búsquedas de manera más eficiente.

Al crear una tabla podemos crear varios índices con las siguientes palabras reservadas:

- **KEY o PRIMARY KEY**: un índice único para el campo de clave primaria.
- **UNIQUE**: un índice único para un campo que sea clave alternativa.
- **INDEX**: un índice con valores repetidos para optimizar búsquedas sobre un campo.

Los índices se pueden crear en el momento de la creación de la tabla añadiendo filas con la opción **INDEX** después de la definición de los campos.

```
CREATE TABLE nombreTabla (  
    Campo1,  
    Campo2,  
    ...  
    CampoN  
    INDEX [nombreIndice] (catecampoI)  
);
```

Se crea un índice sobre el **campo I** y, opcionalmente, le podemos dar un nombre (mejor dejar que el SGBD le de un nombre automáticamente).

Ejemplo 1

Índices en tablas

Creación de la tabla **articulos** y definición de un índice sobre el campo **categoría**.

```
CREATE TABLE articulos (  
    codigo VARCHAR(4) PRIMARY KEY,  
    descripcion VARCHAR(100) UNIQUE,  
    categoria VARCHAR(20),  
    precio DECIMAL(10,2),  
    INDEX (categoria)  
);
```

Añadir índices

Si deseáramos crear un índice con valores repetidos cuando la tabla ya existe, utilizaríamos el comando siguiente:

```
CREATE INDEX index_name ON nombreTabla (col_name1,...);
```

Si deseáramos crear un índice cuando con valores únicos cuando la tabla ya existe, utilizaríamos el comando siguiente:

```
CREATE UNIQUE INDEX index_name ON nombreTabla (col_name1,...);
```

Para añadir una clave primaria a una tabla que no tiene podemos hacerlo con la instrucción **ALTER TABLE**.

```
ALTER TABLE nombreTabla  
ADD PRIMARY KEY (col_name1,...);
```

Eliminar índices

Para eliminar un índice deberemos saber su nombre.

```
DROP INDEX nombreIndice ON nombreTabla;
```

Cuando se define una clave primaria automáticamente se crea un índice único sobre el campo (o campos) que definen dicha clave.

Podemos eliminar el índice asociado a la clave primaria con la instrucción `ALTER TABLE`.

```
ALTER TABLE nombreTabla  
DROP PRIMARY KEY;
```

Restricciones

Restricción de campo clave

La necesidad de identificar un registro unívocamente nos obliga a crear una restricción por el campo clave, ya que este no podrá ser nulo ni tener valores duplicados.

De hecho no se pueden editar los datos de una tabla que no tenga clave primaria, aunque sí se pueden insertar registros.

Las claves (primarias y foráneas) pueden ser:

- **Simles:** formadas por un solo campo
- **Compuestas:** formadas por la concatenación de varios campos

Definir la clave primaria en el momento de crear la tabla

Esta se puede declarar en la misma fila en la que se define un campo (sólo es válido si el campo es simple) o después de la definición de los campos en una línea independiente (válido para todo tipo de claves, simples o compuestas).

Ejemplo 1

Restricciones campo clave tablas

Crear la tabla **articulos** y definir el campo **codigo** como clave primaria.

```
-- La clave primaria se define al lado del campo  
CREATE TABLE articulos (  
    codigo VARCHAR(4) PRIMARY KEY,  
    descripcion VARCHAR(100) ,  
    precio DECIMAL(10,2)  
);  
  
-- La clave primaria se define al final en una fila independiente  
CREATE TABLE articulos (  
    codigo VARCHAR(4),  
    descripcion VARCHAR(100) ,  
    precio DECIMAL(10,2),  
    PRIMARY KEY (codigo)  
);
```

Eliminar la clave primaria

Se puede llevar a cabo con la instrucción `ALTER TABLE` combinada con la opción `DROP`.

Ejemplo 2

Restricciones campo clave tablas

Eliminar la clave primaria de la tabla **articulos**.

```
ALTER TABLE articulos DROP PRIMARY KEY;
```

Añadir la clave primaria a una tabla existente

Se puede añadir la clave primaria con posterioridad a la creación de la tabla con la instrucción **ALTER TABLE** combinada con la opción **ADD**.

Ejemplo 3

Restricciones campo clave tablas

Añadir la clave primaria definida sobre el campo **codigo** a la tabla **articulos**.

```
ALTER TABLE articulos ADD PRIMARY KEY(codigo);
```

Claves compuestas

Ya hemos dicho anteriormente que una clave es compuesta si está formada por la concatenación de varios campos.

Ejemplo 4

Restricciones campo clave tablas

Crear la tabla **revision_itv** donde se guardarán las revisiones de ITV de los vehículos. La clave primaria está formada por la **matrícula** y la **fecha** de matriculación del vehículo.

```
CREATE TABLE revision_itv (  
    matricula VARCHAR(10),  
    fecha DATE,  
    estado VARCHAR(100),  
    PRIMARY KEY (matricula, fecha)  
);
```

Se podría haber creado la tabla en primer lugar y a continuación añadir la clave primaria.

Ejemplo 4b

Restricciones campo clave tablas

```
-- Crear la tabla
CREATE TABLE revision_itv (
    matricula VARCHAR(10),
    fecha DATE,
    estado VARCHAR(100)
);
-- Añadir la clave primaria
ALTER TABLE revision_itv ADD PRIMARY KEY (matricula, fecha);
```

Restricción de campo obligatorio

Cuando un campo no puede tener valores nulos, decimos que es un campo obligatorio. En nuestro ejemplo indicaremos la descripción como campo obligatorio para que ningún artículo tenga la descripción vacía.

Definir campo obligatorio al crear la tabla

Un campo obligatorio se define con la opción `NOT NULL` al lado del campo.

Ejemplo 5

Restricciones campo obligatorio

Crear la tabla **articulos** y definir el campo `descripcion` como obligatorio.

```
CREATE TABLE articulos (
    codigo VARCHAR(4) PRIMARY KEY,
    descripcion VARCHAR(100) NOT NULL,
    precio DECIMAL(10,2)
);
```

Eliminar la restricción de campo obligatorio

Para eliminar la restricción de campo obligatorio utilizamos la instrucción `ALTER TABLE` con la opción `CHANGE`.

Ejemplo 6

Restricciones campo obligatorio

Eliminar la restricción de campo obligatorio del campo `descripcion` de la tabla **articulos**.

```
ALTER TABLE articulos CHANGE descripcion VARCHAR(100) NULL;
```

Añadir la restricción de campo obligatorio en una tabla que ya existe

También se hace con la instrucción `ALTER TABLE` con la opción `CHANGE`.

Ejemplo 7

Restricciones campo obligatorio

Añadir la restricción de campo obligatorio del campo `descripcion` de la tabla **articulos**.

```
ALTER TABLE articulos CHANGE descripcion descripcion VARCHAR(100) NOT NULL;
```

Restricción de campo con valores únicos

Cuando un campo no puede tener valores repetidos, decimos que es un campo único. Cuando se define un campo único automáticamente se crea un índice sobre ese con valores únicos.

Definir un campo único al crear la tabla

Se puede hacer con la opción **UNIQUE** al lado del campo sobre el que se define.

Ejemplo 8

Restricciones campo único

Definir una restricción única sobre el campo **descripcion** en el momento de la creación de la tabla **articulos**.

```
CREATE TABLE articulos (  
  codigo VARCHAR(4) PRIMARY KEY,  
  descripcion VARCHAR(100) NOT NULL UNIQUE,  
  precio DECIMAL(10,2)  
);
```

Eliminar la restricción de campo único

Se lleva a cabo con la instrucción **ALTER TABLE** combinada con la opción **DROP**. La restricción se elimina borrando el índice asociado que se crea cuando se define ésta.

Ejemplo 9

Restricciones campo único

Eliminar la restricción única definida sobre el campo **descripcion** de la tabla **articulos**.

```
ALTER TABLE articulos DROP INDEX descripcion;
```

Añadir la restricción de campo único en una tabla que ya existe

Se lleva a cabo con la instrucción **ALTER TABLE** combinada con la opción **ADD**.

Ejemplo 10

Restricciones campo único

Añadir una restricción única sobre el campo **descripcion** de la tabla **articulos**.

```
ALTER TABLE articulos ADD UNIQUE(descripcion);
```

Restricción de campo por clave ajena

Como ya hemos visto en los esquemas lógicos relacionales obtenidos de los esquemas conceptuales, existen claves ajenas, foráneas o extranjeras que contienen valores de la clave primaria de otra tabla con la que se relacionan.

En un esquema que tiene localidades y provincias, donde indicamos en cada localidad de qué provincia es, podríamos tener las siguientes tablas:

```
CREATE TABLE provincias (  
    cod_prov INT(2) PRIMARY KEY,  
    provincia VARCHAR(40)  
);  
  
CREATE TABLE localidades (  
    cod_loc INT(5) PRIMARY KEY,  
    localidad VARCHAR(100),  
    cod_provincia INT(2)  
);
```

Crear clave ajena al crear la tabla

La clave ajena se puede definir en el momento de crear la tabla declarándola después de la definición de los campos de la tabla.

Ejemplo 11

Restricciones campo por clave ajena

Crear la tabla **localidades** definiendo una clave foránea sobre el campo **cod_provincia** que haga referencia a la tabla **provincias**.

```
CREATE TABLE localidades (  
    cod_loc INT(5) PRIMARY KEY,  
    localidad VARCHAR(100),  
    cod_provincia INT(2),  
    FOREIGN KEY (cod_provincia) REFERENCES provincias (cod_prov)  
);
```

Cuando creamos una relación, y por lo tanto una clave ajena, se crean dos elementos en la BD:

- **Índice:** un índice (en nuestro ejemplo **INDEX(cod_provincia)**) con el nombre del campo para la clave ajena con valores repetidos, ya que puede ser una relación 1 → N y por lo tanto tendrá que repetir valores. En nuestro ejemplo, varias localidades tendrán la misma provincia.
- **Relación:** se asigna a la relación un nombre por defecto formado por el nombre de la tabla, el literal *ibfk* y un numero. En el caso anterior será **localidades_ibfk1**.

Si queremos darle un nombre nosotros utilizaremos la cláusula **CONSTRAINT** :

Ejemplo 12

Restricciones campo por clave ajena

Ejemplo equivalente al anterior en el que se le ha dado el nombre **provincias_cod_prov** a la restricción de clave ajena.


```
CREATE TABLE localidades (
  cod_loc INT(5) PRIMARY KEY,
  localidad VARCHAR(100),
  cod_provincia INT(2),
  CONSTRAINT provincias_cod_prov FOREIGN KEY (cod_provincia) REFERENCES provincias (cod_prov)
);
```

De esta forma se utilizará el valor de **CONSTRAINT** tanto para el índice como para la relación.

Eliminar la restricción de clave ajena

Se utiliza la instrucción **ALTER TABLE** en combinación con la opción **DROP** para este propósito. Si hemos definido la restricción con una cláusula **CONSTRAINT** la utilizaremos.

Ejemplo 13

Restricciones campo por clave ajena

Eliminar la restricción de clave ajena de nombre **provincias_cod_prov**.

```
ALTER TABLE localidades DROP FOREIGN KEY provincias_cod_prov;
```

En caso de no haber utilizado la cláusula **CONSTRAINT** en la definición de la restricción entonces la eliminaremos indicando el nombre que se le ha dado por defecto.

Ejemplo 14

Restricciones campo por clave ajena

Eliminar la restricción de clave ajena definida anónimamente en la tabla **localidades**. Sabemos que sólo hay esta restricción de clave ajena.

```
ALTER TABLE localidades DROP FOREIGN KEY localidades_ibkf_1;
```

También tendremos que decidir si deseamos eliminar el índice existente o no. En caso de desear eliminarlo deberemos indicarlo con la instrucción correspondiente según hayamos indicado **CONSTRAINT** o no. Si no hemos utilizado la opción **CONSTRAINT** entonces el índice tiene el mismo nombre que el campo sobre el que se define.

Ejemplo 15

Restricciones campo por clave ajena

Eliminar el índice asociado a la clave ajena definida en la tabla **localidades** sobre el campo **cod_provincia**.

```
-- Se le había dado un nombre al índice.
ALTER TABLE localidades DROP INDEX provincias_cod_prov;
-- No se le había dado un nombre al índice.
ALTER TABLE localidades DROP INDEX cod_provincia;
```

Añadir la restricción de clave ajena a una tabla existente

Se utiliza la instrucción `ALTER TABLE` en combinación con la opción `ADD` para este propósito.

Ejemplo 16

Restricciones campo por clave ajena

Añadir la una clave ajena sobre el campo `cod_provincia` de la tabla **localidades** que haga referencia a la tabla **provincias**.

```
-- Sin cláusula CONSTRAINT
ALTER TABLE localidades ADD
FOREIGN KEY (cod_provincia) REFERENCES provincias (cod_prov);
-- Con cláusula CONSTRAINT
ALTER TABLE localidades ADD CONSTRAINT provincias_cod_prov
FOREIGN KEY (cod_provincia) REFERENCES provincias (cod_prov);
```

Restricción por borrado o actualización de una clave ajena

Una de las acciones más importantes para mantener la integridad en una base de datos es identificar correctamente la acción a realizar cuando eliminamos o actualizamos el valor de la clave ajena de un registro.

En el caso de vaciar el campo `localidades.cod_provincia` podemos llevar a cabo varias acciones:

- **RESTRICT:** Es el valor por defecto. El servidor MySQL rechazará la operación de eliminación o actualización para la tabla padre (provincias) si hay un valor de clave externa relacionado en la tabla referenciada (localidades), es decir, no se puede en provincias eliminar un registro o cambiar su clave, si existen localidades de esa provincia.
- **NO ACTION:** Pertenece al SQL estándar. En MySQL es equivalente a RESTRICT. Algunos sistemas de base de datos tienen verificaciones diferidas, y NO ACTION es un cheque diferido. En MySQL, las restricciones de clave externa se verifican inmediatamente, por lo que NO ACCIÓN es lo mismo que RESTRICT.
- **SET NULL:** Si en la tabla padre (provincias) se elimina un registro o se actualiza su clave, si hay algún valor de clave externa relacionado en la tabla referenciada (localidades) se actualizará el valor a NULL.
Si especifica una acción SET NULL, asegúrese de que no ha declarado las columnas en la tabla secundaria como NOT NULL.
- **CASCADE:**
 - Al **borrar**, si en la tabla padre (provincias) se elimina un registro, se eliminarán también todos los registros de la tabla referenciada (localidades) que tenga este valor en su clave ajena.
 - Al **actualizar**, si en la tabla padre (provincias) se actualiza la clave de un registro, se actualizarán también todos los registros de la tabla referenciada (localidades) que tenga este valor en su clave ajena con el nuevo valor.

Para indicar estas acciones **añadiremos a la sentencia** `FOREIGN KEY` **alguna de las siguientes cláusulas:**

- `ON DELETE RESTRICT`
- `ON DELETE SET NULL`
- `ON DELETE CASCADE`
- `ON UPDATE RESTRICT`
- `ON UPDATE SET NULL`
- `ON UPDATE CASCADE`

Ejemplo 17

Restricciones por borrado/actualización de clave ajena

Definir una instrucción sobre la tabla **localidades** de manera que al eliminar una provincia se eliminen todas las localidades de esa provincia y que no permita cambiar el identificador de provincia si tiene localidades.

```
ALTER TABLE localidades ADD CONSTRAINT provincias_cod_prov
FOREIGN KEY (cod_provincia) REFERENCES provincias (cod_prov)
ON DELETE CASCADE
ON UPDATE RESTRICT;
```

- **SET DEFAULT:** No disponible para MySQL con el motor de base de datos InnoDB. Si en la tabla padre (**provincias**) se elimina un registro o se actualiza su clave, si hay algún valor de clave externa relacionado en la tabla referenciada (**localidades**) se actualizará el valor por defecto de la clave ajena.

Otras restricciones y valores por defecto

Enteros positivos

Al definir un campo podemos indicar que sólo se aceptarán números positivos con la palabra reservada **UNSIGNED** y asignarle un valor por defecto determinado con la palabra reservada **DEFAULT** si cuando se inserta un registro no se proporciona un valor para dicho campo.

Ejemplo 18

Otras restricciones y valores por defecto

En el momento de la creación de la tabla **articulos**, definid el campo **precio** para que no pueda aceptar valores negativos y que por defecto valga 0.

```
CREATE TABLE articulos (
  codigo VARCHAR(3) PRIMARY KEY,
  descripcion VARCHAR(100) NOT NULL,
  precio DECIMAL(10,2) UNSIGNED DEFAULT 0
);
```

Fecha de alta y de modificación

Cuando deseamos tener en una tabla la fecha de alta y modificación podemos utilizar campos que se creen y actualicen automáticamente.

- Con **DEFAULT CURRENT_TIMESTAMP** un campo obtendrá por defecto el valor de la fecha y hora del sistema.
- Con **ON UPDATE CURRENT_TIMESTAMP** un campo actualizará el valor con la fecha y hora del sistema cada vez que se actualice el registro.

Ejemplo 19

Otras restricciones y valores por defecto

En el momento de creación de la tabla **ejemplo**, definid el campo **fecha_modificacion** para que se actualice con el valor del instante en que se lleve a cabo la actualización de un registro y para que el campo **fecha_alta** se ajuste al instante actual cuando se inserte un registro.

```
CREATE TABLE ejemplo (  
    codigo VARCHAR(2) PRIMARY KEY,  
    fecha_alta DATETIME DEFAULT CURRENT_TIMESTAMP,  
    fecha_modificacion DATETIME ON UPDATE CURRENT_TIMESTAMP  
);
```

Rellenado a ceros

El relleno con ceros a la izquierda se lleva a cabo sobre campos de tipo `INT`. Para ello utilizamos la opción `ZEROFILL`.

Ejemplo 20

Otras restricciones y valores por defecto

En el momento de la creación de la tabla **ejemplo** definid el campo `codigo` para que se rellene con ceros a la izquierda.

```
CREATE TABLE ejemplo (  
    codigo INT(4) ZEROFILL PRIMARY KEY,  
    descripcion VARCHAR(50) NOT NULL  
);  
/* El valor del código 1 se insertará relleno con ceros a la izquierda  
para obtener el valor con 4 dígitos. */  
INSERT INTO ejemplo (codigo, descripcion) VALUES ('1','registro uno');
```

Clave primaria automática - Autoincremento

Cuando tenemos una entidad donde almacenamos registros pero en el esquema conceptual del modelo entidad-relación no teníamos una clave primaria, tenemos que añadir un atributo `codigo` (o cualquier otro nombre) para disponer de una clave primaria.

Lo normal es que esta información no se la pidamos al usuario, ya que en el sistema a analizar no aparecía esta información, por lo que se suele dejar que sea el sistema el que genere este código automáticamente con la palabra reservada

`AUTO_INCREMENT`.

Ejemplo 22

Otras restricciones y valores por defecto

Crear la tabla **ejemplo** de manera que la clave primaria sea un campo autoincrementado.

```
CREATE TABLE ejemplo (  
    codigo INT(4) AUTO_INCREMENT PRIMARY KEY,  
    descripcion VARCHAR(50) NOT NULL  
);  
-- No especificamos el campo codigo ya que este es autoincrementado  
INSERT INTO ejemplo(descripcion) VALUES ('registro uno');
```

En **phpMyAdmin**, seleccionando la tabla y luego pulsando en la pestaña operaciones, podemos ver y cambiar el valor siguiente de `AUTO_INCREMENT`.

Aviso

No proporcionar nunca valores de los campos autoincrementados en las instrucciones de inserción ya que esto nos puede conducir a errores difíciles de tratar.

Restricciones avanzadas

Campos calculados

Una solución para los campos calculados es no crearlos en las tablas sino en las consultas con funciones de MySQL.

Este tipo de cálculos y funciones los veremos en la siguiente unidad didáctica donde trabajaremos el lenguaje de consultas SQL con las instrucciones **SELECT**.

De todas formas, para aquellos que deseen ir practicando, podéis consultar las funciones disponibles en:

<https://dev.mysql.com/doc/refman/8.0/en/func-op-summary-ref.html>

Filtro de valores válidos

En el lenguaje de SQL existe la palabra reservada **CHECK** para indicar la condición que debe cumplir un valor de un campo para ser válido.

Ejemplo 23

Restricciones avanzadas

Crear la tabla **notas** de manera que la **nota** del alumno esté entre 0 y 10.

```
CREATE TABLE notas (  
  nia VARCHAR(8),  
  asignatura VARCHAR(15),  
  nota DECIMAL (6,2),  
  CONSTRAINT check_nota CHECK ( (nota >= 0) AND (nota <= 10) )  
);
```

Transformaciones antes de grabar

Al igual que en el apartado anterior **CHECK** nos permite realizar estos cambios.

Ejemplo 24

Restricciones avanzadas

Deseamos que en el campo **asignatura** se almacenen los valores en mayúscula independientemente de como los proporcione el usuario. Para ello utilizaremos la función **UPPER()** dentro de una restricción **CHECK**.

```
CREATE TABLE notas (  
  nia VARCHAR(8),  
  asignatura VARCHAR(15),  
  nota DECIMAL (6,2),  
  CONSTRAINT check_asignatura CHECK ( asignatura = UPPER(asignatura) )  
);
```

Aunque esta definición no da ningún error en MySQL, desafortunadamente esta funcionalidad no está implementada a día de hoy. El problema se puede solventar definiendo un procedimiento especial llamado **trigger** o disparador. En una unidad

didáctica posterior se tratará en profundidad el trabajo con triggers.

En cualquier caso, a título informativo, se mostrará el código de este procedimiento que en nuestro caso tendrá la función de interceptar el valor de la asignatura que se quiere insertar y sustituirá el valor proporcionado por el mismo pero pasado a mayúsculas.

Anexo ejemplo 24

Restricciones avanzadas

Creación del **trigger** que soluciona el paso a mayúsculas del nombre de las asignaturas.

```
/* Crear el disparador */
DELIMITER //
CREATE TRIGGER notas_before_insert
BEFORE INSERT ON notas
FOR EACH ROW
BEGIN
    SET NEW.asignatura = UPPER(NEW.asignatura);
END //
DELIMITER ;
```

Para eliminar un **TRIGGER** disponemos de la instrucción **DROP**.

Anexo ejemplo 24b

Restricciones avanzadas

Eliminar el **trigger** creado en el ejemplo anterior.

```
DROP TRIGGER notas_before_insert;
```

Tablas a partir de consultas

Existe la posibilidad de crear tablas a partir de consultas de otras tablas.

La sintaxis es muy sencilla, pues si tenemos una consulta del tipo:

```
SELECT campo1, campo2, ... campoK
FROM nombretabla
[WHERE condicion];
```

Podríamos obtener una tabla con la siguiente instrucción SQL:

```
CREATE TABLE nuevatabla
SELECT campo1, campo2, ... campoK
FROM nombretabla
[WHERE condicion];
```

Pero eso sí, la crea **sin clave primaria, ni restricciones ni índices**.

Ejemplo 1

Tablas a partir de consultas

Crear un duplicado de la tabla **provincias** de nombre **provincias2**.

```
CREATE TABLE provincias2
SELECT * FROM provincias;
```

Vistas

Una **vista** es una consulta que se presenta como una tabla (virtual) a partir de un conjunto de tablas en una base de datos relacional.

Las vistas tienen la misma estructura que una tabla: filas y columnas. La única diferencia es que sólo se almacena de ellas la definición, no los datos. Los datos que se recuperan mediante una consulta a una vista se presentarán igual que los de una tabla. De hecho, si no se sabe que se está trabajando con una vista, nada hace suponer que es así. Al igual que sucede con una tabla, se pueden insertar, actualizar, borrar y seleccionar datos en una vista. Aunque siempre es posible seleccionar datos de una vista, en algunas condiciones existen restricciones para realizar el resto de las operaciones sobre vistas.

Una vista se especifica a través de una expresión de consulta (una sentencia **SELECT**) que la calcula y que puede realizarse sobre una o más tablas. Sobre un conjunto de tablas relacionales se puede trabajar con un número cualquiera de vistas.

La mayoría de los SGBD soportan la creación y manipulación de vistas. La sintaxis es muy sencilla, pues si tenemos una consulta de tipo:

```
SELECT campo1, campo2, ... campoK
FROM nombretabla
[WHERE condicion];
```

Podríamos obtener una vista con la sentencia **CREATE VIEW** de la siguiente manera:

```
CREATE VIEW nuevaVista AS
SELECT campo1, campo2, ... campoK
FROM nombretabla
[WHERE condicion];
```

Ejemplo 1

Vistas

Crear una vista de nombre **personas1** a partir de la tabla **personas** de manera que sólo contenga las provincias de código igual a 1.

```
CREATE VIEW personas1 AS
SELECT * FROM personas WHERE cod_provincia = 1;
```

Vistas para aplicar restricciones

Cuando creamos una vista de una tabla con restricciones de tipo **CHECK**, por defecto, las inserciones en la vista no tendrán en cuenta las restricciones. Esto se puede solucionar añadiendo la cláusula **WITH CHECK OPTION** después de la cláusula **WHERE** (si hay).

```
CREATE VIEW nuevatabla AS
SELECT campo1, campo2, ... campoK
FROM nombretabla
[WHERE condicion]
WITH CHECK OPTION;
```

También se puede crear una vista con restricciones **CHECK** aunque la tabla sobre la que se realiza no las tenga.

Ejemplo 2

Vistas

Tenemos una tabla denominada **notas_data**

```
CREATE TABLE notas_data (
  nia VARCHAR(8),
  asignatura VARCHAR(15),
  nota DECIMAL (6,2)
);
```

Crear una vista sobre esta tabla con la restricción de valor de **nota** entre 0 y 10.

```
CREATE VIEW notas AS
SELECT nia, asignatura, nota FROM notas_data WHERE (nota >= 0) and (nota <= 10)
WITH CHECK OPTION;
```

Vistas para aplicar transformaciones o mostrar campos calculados

El siguiente ejemplo aclara este concepto.

Ejemplo 3

Supongamos que tenemos una tabla denominada **notas_data**

```
CREATE TABLE notas (
  nia VARCHAR(8),
  asignatura VARCHAR(15),
  nota1 DECIMAL (6,2),
  nota2 DECIMAL (6,2)
);
```

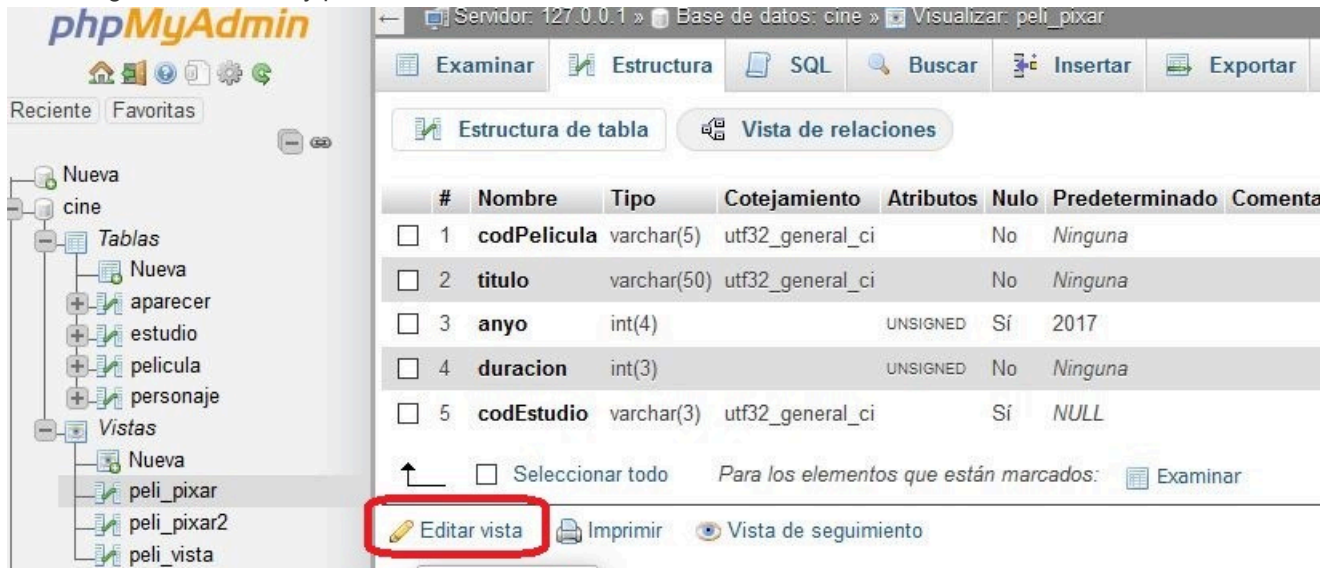
Crear una vista que tenga el campo **asignatura** en mayúsculas y un campo calculado adicional que sea la nota media (**promedio**) de **nota1** y **nota2** con una precisión de un decimal.


```
CREATE VIEW notas_view AS
SELECT nia, UPPER(asignatura) as asignatura_upper, nota1, nota2, round((nota1+nota2)/2, 1) as promedio FROM notas;
```

Este sistema nos obligaría a editar los datos en la tabla **notas** pero visualizar el resultado con la vista **notas_view**.

Vistas con phpMyAdmin

Una vez hemos creado una vista, podremos editarla si lo deseamos. Para ello, si utilizamos phpMyAdmin, pulsaremos sobre la vista, luego en *Estructura* y posteriormente en *Editar vista*:



The screenshot shows the phpMyAdmin interface. On the left, the database structure is visible, with the 'Vistas' (Views) folder expanded, showing 'peli_pixar', 'peli_pixar2', and 'peli_vista'. The main panel displays the 'Estructura de tabla' (Table Structure) view for the 'peli_pixar' view. The table structure is as follows:

#	Nombre	Tipo	Cotejamiento	Atributos	Nulo	Predeterminado	Comenta
☐ 1	codPelicula	varchar(5)	utf32_general_ci		No	Ninguna	
☐ 2	titulo	varchar(50)	utf32_general_ci		No	Ninguna	
☐ 3	anyo	int(4)		UNSIGNED	Sí	2017	
☐ 4	duracion	int(3)		UNSIGNED	No	Ninguna	
☐ 5	codEstudio	varchar(3)	utf32_general_ci		Sí	NULL	

Below the table structure, there are buttons for 'Seleccionar todo' (Select all), 'Examinar' (Examine), 'Imprimir' (Print), and 'Vista de seguimiento' (Follow view). The 'Editar vista' (Edit view) button is highlighted with a red box.